

序列化概述

序列化：将一个对象转化成字节序列，变成一个 **可传输、可储存** 的状态

- 序列化后的对象，可以在网络上进行传输，也可以存储到本地
- 对象流 `ObjectOutputStream` 的 `readObject`、`writeObject` 也要求对象可序列化

反序列化：将字节序列转化为一个对象

序列化的方式：

- 实现 **Serializable** 接口（Java自带）：通过 `ObjectOutputStream` / `ObjectInputStream` 将对象内容转化为 IO 流进行传输（网络传输、持久化到本地）
- 实现 **Parcelable** 接口（Android专用）：通过 **Parcel** 将对象内容进行分解/重组，按照 Parcel 的方式存储在 Parcel 内部进行传输（Binder 通讯）

Serializable 和 Parcelable 的比较

Serializable 通过 **反射** 的方式调用 `readObject()` / `writeObject()` 方法，将实例对象写入到 `ObjectInputStream` / `ObjectOutputStream` 中

- IO 流可以写入到文件，因此数据可以进行持久化保存
- 内存开销大、效率低

Parcelable 调用 **readFromParcel()** /

writeToParcel() 方法，将实例对象分解/重组/并按照一定规则写入到 **Parcel** 对象中

- 数据仅仅存在内存中，因此数据无法进行持久化保存
- 内存开销小、效率高

Serializable 是 IO 流的序列化（将对象转化为字节流，可以进行本地存储），**Parcelable** 是 **Binder** 通讯的序列化（将对象存储在 **Parcel** 内部）

Serializable – 简单至上

- Serializable 是利用 **反射** 的方式实现的，效率低
- Serializable 的创建机制会在创建的过程中，产生大量的临时变量，从而 **引发频繁的 GC**
- 实现方式简单

Parcelable – 性能至上

- Parcelable 效率高：我们清楚的知道了序列化的过程
- Parcelable 序列化原理：**将一个完整的对象进行分解**，分解后的每一个部分都可以用来传输、存储
- 实现方式相对复杂

使用场景的差别

- 如果仅仅是将数据序列化到内存中，Parcelable 效率高，推荐 Parcelable
- 在将数据保存到磁盘上的时候（持久性），Parcelable 在外界发生变化的时候，无法保证数据的持续性（需要更改它的序列化和反序列化过程），推荐 Serializable

实现方式的差别

- Serializable 只需要当前类继承 Serializable 即可：相当于对当前对象打了个标记，系统会自动将其序列化
- Parcelable 不仅要继承 Parcelable，还需要我们手动进行序列化步骤
 - 重写 2 个方法：describeContents、writeToParcel
 - 创建一个 CREATOR 静态变量，这个变量要继承 Parcelable.Creator 接口

Serializable

参考：[【序列化/反序列化，我忍你很久了】](#)

自动序列化：一个对象，如果实现了 `Serializable` 接口，就可以被序列化，Java 这种序列化方式给开发者提供了很多便利，我们不必关心具体的序列化过程，只要这个类实现了 `Serializable` 接口，这个类的所有属性和方法都会被序列化

- **递归 `Serializable`**：一个类如果实现了 `Serializable` 接口，那么这个类里所有的成员变量都要去实现 `Serializable` 或者是基本数据类型
- **基本数据类型、枚举、数组、`Class`、`String`** 本身就可以进行序列化（特有的序列化方法）
- **`String`、`ArrayList`、`HashMap`** 本身都实现了 `Serializable` 接口（重写了方法）

手动序列化：重写 `writeObject()` 和 `readObject()` 两个方法，可以对我们的序列化过程进行控制

- **`readResolve()`** 返回自定义的对象，代替序列化过程
单例的序列化过程，要求序列化前后，是同一个对象，这个时候我们就可以重写 `readResolve()` 方法返回原来的单例对象
Enum 枚举项的序列化过程，其实就是重写了该方法，它可以保证序列化前后是同一个单例对象

`Serializable` 本质是采用 **`ObjectInputStream` / `ObjectOutputStream`** 实现的

2 个特殊情况

transient：被该修饰符修饰的属性，不会参与序列化过程

- 一旦变量被 transient 修饰，该变量就不再是持久化的一部分，在序列化后将无法获得访问
- transient 关键字，只能修饰变量，而无法修饰方法和类
- 会参与序列化过程，但是值永远都是 0

static：被该修饰符修饰的属性，始终不参与序列化过程

- 静态变量属于类，而不是对象
- 如果我们手动实现序列化过程，那么 static 关键字是可以被序列化的

Externalizable 整个类不会被序列化

Serializable

Serializable 有 2 种使用方式

1. 实现一个空接口

- 仅仅起到了一个 “打标记” 的作用，但是它要求 **所有的属性都可以序列化**
- 底层通过 **“反射”** 的方式获取字段值，然后调用 `ObjectOutputStream.write()` 实现的

- 通常配合 `transient` 使用
- 注意如果是一个非静态内部类，它默认持有外部类的一个引用，如果外部类不可序列化会报错

2. 实现 `writeObject()\readObject()` 函数：反射调用

`writeObject()/readObject()`

- `writeObject(ObjectOutput out)`：序列化
- `readObject(ObjectInput in)`：反序列化
- 属性可以是一些 “非Serializable” 的对象

Serializable 本身不包含 `writeObject()`、`readObject()` 两个方法，这 2 个方法由实现了 Serializable 接口的类自定义

- 这 2 个方法必须是 `private`：因为它负责对象内部状态的读写，不应该被外部调用

```
public interface Serializable {  
    private static final long serialVersionUID =  
    7382351359868556980L;  
    private void readObject(ObjectInputStream ois) throws  
    IOException, ClassNotFoundException { }  
    private void writeObject(ObjectOutputStream oos) throws  
    IOException { }  
}
```

Externalizable

Externalizable：所有属性都不会被序列化，需要我们重写 writeExternal、readExternal 两个方法进行序列化

- writeExternal(ObjectOutput out)：序列化
- readExternal(ObjectInput in)：反序列化

transient：单个字段不被序列化

Externalizable 继承自 Serializable

Serializable 的 writeObject()、readObject() 是为了 手动控制 序列化过程；

Externalizable 的 writeExternal()、readExternal() 是为了 选择性 序列化

```
public interface Externalizable extends java.io.Serializable
{
    void writeExternal(ObjectOutput out) throws IOException;
    void readExternal(ObjectInput in) throws IOException,
    ClassNotFoundException;
}
```

序列化、反序列化代码

Serializable

实现 `Serializable` 接口，并重写 `readObject()`、`writeObject()` 两个方法

```
public class Person implements Serializable{
    private static final long serialVersionUID =
7382351359868556980L;
    private void readObject(ObjectInputStream ois) throws
IOException, ClassNotFoundException { }
    private void writeObject(ObjectOutputStream oos) throws
IOException { }
}
```

序列化

序列化对象的时候，需要先让该对象实现 `Serializable` 接口，接着调用 `ObjectOutputStream.writeObject()` 进行序列化

1. 序列化到本地: `ObjectOutputStream + FileOutputStream`
2. 序列化到内存: `ObjectOutputStream + ByteArrayOutputStream`

```
public static void serialize( ) throws IOException {
    Student student = new Student();
    student.setName("CodeSheep");
    student.setAge(18);
    student.setScore(1000);

    ObjectOutputStream objectOutputStream =
```



```
        new ObjectOutputStream( new FileOutputStream( new
File("student.txt") ) );
        objectOutputStream.writeObject(student);
        objectOutputStream.close();

        System.out.println("序列化成功! 已经生成student.txt文件");

        System.out.println("=====
=====");
    }
}
```

反序列化

反序列化对象的时候, 调用

ObjectInputStream.readObject() 进行

1. 从本地读取: **ObjectInputStream + FileInputStream**
2. 从内存读取: **ObjectInputStream +
ByteArrayInputStream**

```
public static void deserialize() throws IOException,
ClassNotFoundException {
    ObjectInputStream objectInputStream =
        new ObjectInputStream( new FileInputStream( new
File("student.txt") ) );
    Student student = (Student)
objectInputStream.readObject();
    objectInputStream.close();

    System.out.println("反序列化结果为: ");
    System.out.println( student );
}
```

手动序列化 - readObject()

1. 自动序列化：只需要当前类继承 **Serializable** 即可
 - 这个方式，相当于对当前对象打了个标记，系统会自动将其序列化
 - Serializable 要求当前类的所有成员变量，都实现了 Serializable 接口（是一个递归的过程）
2. 手动序列化：重写 **readObject()**、**writeObject()** 两个方法
 - 这个方式，相当于手动进行序列化
 - 当前类的成员变量，无需实现 Serializable 接口，也可以进行序列化
 - readObject 表示从序列化输入流中读取对象

- writeObject 表示将对象存入序列化输出流中
- readObject 和 writeObject 的读写顺序要一致，且其调用的读写方法也要一致（writeBoolean 不能用 readObject 来接收）

```
public class Person implements Serializable{
    private static final long serialVersionUID =
7382351359868556980L;
    private void readObject(ObjectInputStream ois) throws
IOException, ClassNotFoundException { }
    private void writeObject(ObjectOutputStream oos) throws
IOException { }
}
```

Cookie 的序列化

mCookie 表示原生的对象，mClientCookie 表示序列化后的对象

```
public class SerializableCookie implements Serializable {

    private transient Cookie mCookie; //
    transient 表示不需要进行序列化
    private transient Cookie mClientCookie;

    public SerializableCookie(Cookie cookie) {
        mCookie = cookie;
    }

    public Cookie getCookie() {
```

```
        return mClientCookie==null ? mCookie : mClientCookie;
    }
}
```

```
    private void writeObject(ObjectOutputStream oos) throws
IOException {
        oos.writeObject(mCookie.name());
        oos.writeObject(mCookie.value());
        oos.writeObject(mCookie.domain());
        oos.writeObject(mCookie.path());
        oos.writeLong(mCookie.expiresAt());
        oos.writeBoolean(mCookie.hostOnly());
        oos.writeBoolean(mCookie.httpOnly());
        oos.writeBoolean(mCookie.secure());
        oos.writeBoolean(mCookie.persistent());
    }
}
```

```
    private void readObject(ObjectInputStream ois) throws
IOException, ClassNotFoundException {
        String name = (String) ois.readObject();
        String value = (String) ois.readObject();
        String domain = (String) ois.readObject();
        String path = (String) ois.readObject();
        Long expiresAt = ois.readLong();
        Boolean hostOnly = ois.readBoolean();
        Boolean httpOnly = ois.readBoolean();
        Boolean secure = ois.readBoolean();
        Boolean persistent = ois.readBoolean();
        Cookie.Builder cookieBuilder = new Cookie.Builder()
            .name(name)
            .value(value)
            .path(path)
            .expiresAt(expiresAt);
    }
}
```

```
        cookieBuilder = hostOnly ?
cookieBuilder.hostOnlyDomain(domain) :
cookieBuilder.domain(domain);
        cookieBuilder = httpOnly ? cookieBuilder.httpOnly() :
cookieBuilder;
        cookieBuilder = secure ? cookieBuilder.secure() :
cookieBuilder;
        mClientCookie = cookieBuilder.build();
    }
}
```

约束性加强 - defaultReadObject()

手动序列化的一个好处就是可以进行 **约束性加强（校验结果）**：
对我们的序列化过程进行一些控制

- 序列化和反序列化的过程其实是有漏洞的，所以我们需要一个受控的对象序列化过程
- 因为从序列化到反序列化是有中间过程的，如果被别人拿到了中间字节流，然后加以伪造或者篡改，那反序列化出来的对象就会有一定风险了
- 自动序列化无法对结果进行校验

defaultReadObject()、**defaultWriteObject()** 表示调用默认的方法进行序列化/反序列化，一般在 **readObject()**、**writeObject()** 内部使用

- 我们可以在 readObject()、writeObject() 内部对结果进行校验
- 它其实是对 非 static、非 transient 的变量进行序列化，一般在 readObject()、writeObject() 内部使用

```
public class MySerializable implements Serializable {

    private static final String TAG = "MySerializable";

    private String name;
    private int age;

    private void readObject(ObjectInputStream in) throws
IOException, ClassNotFoundException {
        // 3. defaultReadObject()
        in.defaultReadObject();
        Log.d(TAG, "readObject: name=" + name + " age=" + age);
        // name=小米 age=18

        // 4. 约束性加强: 在这里我们可以对其进行校验, 如果不符合要求修改
        age = 20;
        if( 0 > age || 100 < age ) {
            throw new IllegalArgumentException("年龄只能在0到100之
间! ");
        }
    }

}

public static void test() {
    try {
```

```
MySerializable obj = new MySerializable();  
obj.name = "小米";  
obj.age = 18;
```

// 1. 序列化到内存

```
ByteArrayOutputStream baos = new ByteArrayOutputStream();  
ObjectOutputStream oos = new ObjectOutputStream(baos);  
oos.writeObject(obj);
```

// 2. 反序列化获取

```
ByteArrayInputStream bais = new  
ByteArrayInputStream(baos.toByteArray());  
ObjectInputStream ois = new ObjectInputStream(bais);  
MySerializable obj2 = (MySerializable) ois.readObject();  
Log.d(TAG, "test: name=" + obj2.name + " age=" +  
obj2.age); // name=小米 age=20  
} catch (Exception e) {  
    e.printStackTrace();  
}  
}
```

单例序列化 - readResolve()

如果按照正常的序列化和反序列方式对单例进行操作，那么很明显
序列化前后是 2 个不同的对象（单例）

- 我们需要重写 `readResolve()` 方法，直接返回单例对象，来规避这个问题
- 如果这个单例是一个枚举类，那么序列化前后依然是同一个对

象

`readResolve()`：如果重写了这个方法，那么在反序列化从流中读取对象的时候，`readResolve()` 方法会代替 `readObject()` 来获取对象 `Object`

```
private Object readResolve() throws ObjectStreamException {  
    return SingletonHolder.singleton;  
}
```

案例

1. 这里我们采用静态内部类的方式实现单例

```
public class Singleton implements Serializable {  
  
    private static final long serialVersionUID =  
-1576643344804979563L;  
  
    private Singleton() {  
  
    }  
  
    public static synchronized Singleton getSingleton() {  
        return SingletonHolder.singleton;  
    }  
  
    private static class SingletonHolder {  
        private static final Singleton singleton = new  
Singleton();  
    }  
}
```



```
}
```

```
}
```

2. 然后写一个验证主函数，2 个单例竟然不相等！

```
public class Test2 {

    public static void main(String[] args) throws IOException,
ClassNotFoundException {
        // 将单例对象先序列化到文本文件singleton.txt中
        ObjectOutputStream outputStream =
            new ObjectOutputStream(
                new FileOutputStream( new File("singleton.txt") )
            );
        outputStream.writeObject( Singleton.getSingleton()
    );
        outputStream.close();

        // 将文本文件singleton.txt中的对象反序列化为singleton1
        ObjectInputStream inputStream =
            new ObjectInputStream(
                new FileInputStream( new File("singleton.txt") )
            );
        Singleton singleton1 = (Singleton)
inputStream.readObject();
        inputStream.close();

        Singleton singleton2 = Singleton.getSingleton();

        // 运行结果竟打印 false !
        System.out.println( singleton1 == singleton2 );
    }
}
```

```
}
```

```
}
```

标识符 – serialVersionUID

在实现 Serializable 接口的时候，我们通常会定义一个 **serialVersionUID**，那么这个究竟是干什么的呢？

- serialVersionUID 是 **序列化前后的一个唯一标识**
 - 只有 2 者一致，才能进行序列化和反序列化，否则会报错 `InvalidClassException`
- 如果没有人为的进行显示定义，那么 **编译器会自动生成一个**
 - 当我们的类发生变化的时候，这个自动生成的 `serialVersionUID` 就会发生变化
- **建议手动申明**，这样在类发生变化的时候，依然可以进行反序列化

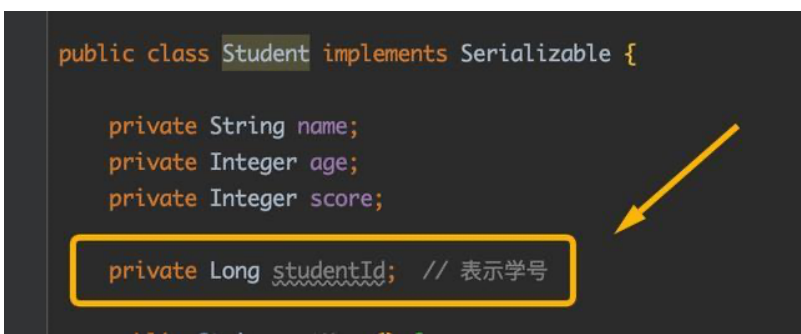
```
private static final long serialVersionUID =  
-4392658638228508589L;
```

案例

1. 我们首先调用 `serialize()` 方法，将一个 `Student` 对象序列化到本地磁盘上的 `student.txt` 文件中

```
public static void serialize() throws IOException {  
    Student student = new Student();  
    student.setName("CodeSheep");  
    student.setAge( 18 );  
    student.setScore( 100 );  
  
    ObjectOutputStream objectOutputStream =  
        new ObjectOutputStream( new FileOutputStream( new  
File("student.txt") ) );  
    objectOutputStream.writeObject( student );  
    objectOutputStream.close();  
}
```

2. 接下来我们在 Student 类里面动点手脚，比如在里面再增加一个名为 studentID 的字段，表示学生学号



```
public class Student implements Serializable {  
  
    private String name;  
    private Integer age;  
    private Integer score;  
  
    private Long studentId; // 表示学号  
  
}
```

3. 这时候，我们拿刚才已经序列化到本地的 student.txt 文件，还用如下代码进行反序列化，试图还原出刚才那个 Student 对象

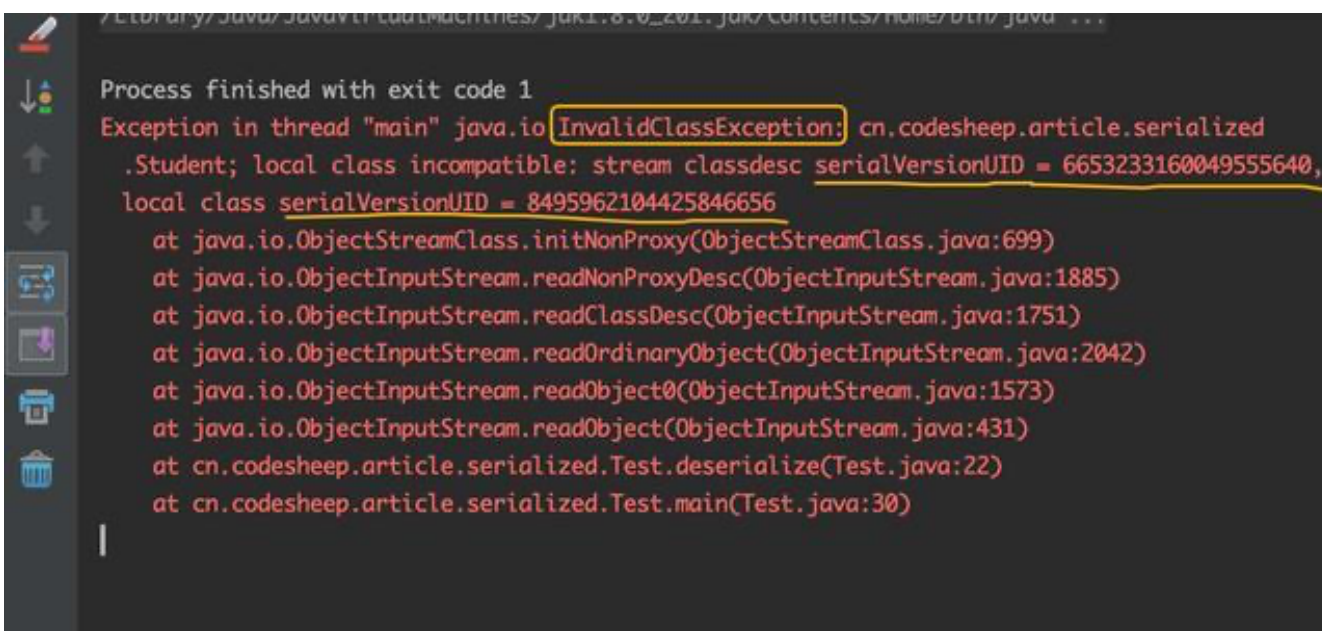
```

public static void deserialize( ) throws IOException,
ClassNotFoundException {
    ObjectInputStream objectInputStream =
        new ObjectInputStream( new FileInputStream( new
File("student.txt") ) );
    Student student = (Student)
objectInputStream.readObject();
    objectInputStream.close();

    System.out.println("反序列化结果为: ");
    System.out.println( student );
}

```

4. 运行发现报错了，并且抛出了 `InvalidClassException` 异常：序列化前后的 `serialVersionUID` 号码不兼容



```

Process finished with exit code 1
Exception in thread "main" java.io.InvalidClassException: cn.codesheep.article.serialized
.Student; local class incompatible: stream classdesc serialVersionUID = 6653233160049555640,
local class serialVersionUID = 8495962104425846656
    at java.io.ObjectStreamClass.initNonProxy(ObjectStreamClass.java:699)
    at java.io.ObjectInputStream.readNonProxyDesc(ObjectInputStream.java:1885)
    at java.io.ObjectInputStream.readClassDesc(ObjectInputStream.java:1751)
    at java.io.ObjectInputStream.readOrdinaryObject(ObjectInputStream.java:2042)
    at java.io.ObjectInputStream.readObject0(ObjectInputStream.java:1573)
    at java.io.ObjectInputStream.readObject(ObjectInputStream.java:431)
    at cn.codesheep.article.serialized.Test.deserialize(Test.java:22)
    at cn.codesheep.article.serialized.Test.main(Test.java:30)

```

源码

序列化最终会调用到 `ObjectOutputStream.writeObject()` / `readObject()`，我们从这里开始跟进

1. 根据 obj 的类型分别调用不同的方法进行序列化：这里 `Serializable` 的标记作用起作用了（String、Class、数组、枚举都有自己的序列化方法）
2. 分别调用 `writeSerialData()`、`writeExternalData()` 对 `Serializable`、`Externalizable` 进行序列化
3. 通过 `反射` 的方式调用 `readObject()`、`writeObject()` 进行序列化

```
// ObjectOutputStream
public final void writeObject(Object obj) throws IOException
{
    if (enableOverride) {
        writeObjectOverride(obj);
        return;
    }

    writeObject0(obj, false);
    ...
}

private void writeObject0(Object obj, boolean unshared)
throws IOException
{
    ...
    if (obj instanceof Class) {
// Class
        writeClass((Class) obj, unshared);
    }
}
```

```

    } else if (obj instanceof ObjectOutputStreamClass) {
// ObjectOutputStream
        writeClassDesc((ObjectStreamClass) obj, unshared);
    } else if (obj instanceof String) {
// String
        writeString((String) obj, unshared);
    } else if (cl.isArray()) {
// 数组
        writeArray(obj, desc, unshared);
    } else if (obj instanceof Enum) {
// 枚举
        writeEnum((Enum<?>) obj, desc, unshared);
    } else if (obj instanceof Serializable) {
// Serializable
        writeOrdinaryObject(obj, desc, unshared);
    } else {
        if (extendedDebugInfo) {
// 抛异常
            throw new NotSerializableException(
                cl.getName() + "\n" + debugInfoStack.toString());
        } else {
            throw new NotSerializableException(cl.getName());
        }
    }
    ...
}

```

```

private void writeOrdinaryObject(Object obj,
                                ObjectOutputStream desc,
                                boolean unshared)
    throws IOException
{

```

```

if (extendedDebugInfo) {
    debugInfoStack.push(
        (depth == 1 ? "root " : "") + "object (class \"" +
        obj.getClass().getName() + "\", " + obj.toString() +
        ")");
}
try {
    desc.checkSerialize();

    bout.writeByte(TC_OBJECT);
    writeClassDesc(desc, false);
    handles.assign(unshared ? null : obj);
    if (desc.isExternalizable() && !desc.isProxy()) {
        writeExternalData((Externalizable) obj);
// writeExternalData
    } else {
        writeSerialData(obj, desc);
// writeSerialData
    }
} finally {
    if (extendedDebugInfo) {
        debugInfoStack.pop();
    }
}
}

```

```

private void writeSerialData(Object obj, ObjectOutputStream
desc)
    throws IOException
{
    ObjectOutputStream.ClassDataSlot[] slots =
desc.getClassDataLayout();

```

```

for (int i = 0; i < slots.length; i++) {
    ObjectOutputStream slotDesc = slots[i].desc;
    if (slotDesc.hasWriteObjectMethod()) {
// writeObjectMethod
        PutFieldImpl oldPut = curPut;
        curPut = null;
        SerialCallbackContext oldContext = curContext;

        if (extendedDebugInfo) {
            debugInfoStack.push(
                "custom writeObject data (class \"" +
                slotDesc.getName() + "\"");
        }
        try {
            curContext = new SerialCallbackContext(obj,
slotDesc);
            bout.setBlockDataMode(true);
            slotDesc.invokeWriteObject(obj, this);
//
invoke 反射
            bout.setBlockDataMode(false);
            bout.writeByte(TC_ENDBLOCKDATA);
        } finally {
            curContext.setUsed();
            curContext = oldContext;
            if (extendedDebugInfo) {
                debugInfoStack.pop();
            }
        }

        curPut = oldPut;
    } else {
        defaultWriteFields(obj, slotDesc);
    }
}

```



```

    }
}
}

public class ObjectOutputStreamClass implements Serializable {

    private ObjectOutputStream(final Class<?> cl) {
        ...
        if (externalizable) {
            cons = getExternalizableConstructor(cl);
        } else {
            cons = getSerializableConstructor(cl);
            writeObjectMethod = getPrivateMethod(cl, "writeObject",
// writeObject()
                                                    new Class<?>[] {
ObjectOutputStream.class },
                                                    Void.TYPE);
            readObjectMethod = getPrivateMethod(cl, "readObject",
// readObject()
                                                    new Class<?>[] {
ObjectInputStream.class },
                                                    Void.TYPE);
            readObjectNoDataMethod = getPrivateMethod( cl,
"readObjectNoData",
                                                    null,
Void.TYPE);
            hasWriteObjectData = (writeObjectMethod != null);
        }
        writeReplaceMethod = getInheritableMethod(
            cl, "writeReplace", null, Object.class);
        readResolveMethod = getInheritableMethod(
            cl, "readResolve", null, Object.class);
    }
}

```

```
...
}

void invokeWriteObject(Object obj, ObjectOutputStream out)
{
    ...
    writeObjectMethod.invoke(obj, new Object[]{ out });
// invoke
    ...
}
}
```

Parcelable

Parcelable 的实现方式

- **writeToParcel()** 往 Parcel 中写
- **readFromParcel()** 从 Parcel 中读，虽然没有这个方法，但是需要我们重写
- **createFromParcel()** 从 Parcel 中创建
- **describeContents()**：当前对象是否包含 "文件描述符" 或者其他一些需要系统处理的内容

0 不包含文件描述符

1 包含文件描述符

- **Creator**

Bundle 和 **Intent** 都是 Parcelable 的子类

- Parcelable、Intent、Bundle 都会写入到 Parcel 中（Parcel 内部实现的），然后进行 Binder 通讯
- AIDL 传递 Parcelable 数据的时候，同样也是先写入到 Parcel 中（自动生成），然后再进行 Binder 通讯

注意：Parcel 写的顺序和读的顺序必须一致

```
public class Intent implements Parcelable, Cloneable;
public final class Bundle extends BaseBundle implements
Cloneable, Parcelable;

public interface Parcelable {

    // 一般我们返回 0 即可
    // 但是如果 Object 包含了一个文件描述符 file descriptor, 那么必须返回
    CONTENTS_FILE_DESCRIPTOR=1
    public int describeContents();

    // 从 Parcel 中创建
    public interface Creator<T> {

        public T createFromParcel(Parcel source);

        public T[] newArray(int size);

    }
}
```

```
// 往 Parcel 中写
public void writeToParcel(Parcel dest, int flags);

// 从 Parcel 中读, 虽然没有这个方法, 但是需要我们重写
public void readFromParcel(Parcel dest);

}
```

实现

1. 重写 `describeContents()`
 - 默认返回 0 即可
2. 重写 `writeToParcel()`
 - 将类中的数据, 都写入 `Parcel` 容器中
3. 重写 `readFromParcel()`
 - 从 `Parcel` 中获取实例对象
4. 创建静态内部对象 `CREATOR`
 - 名称必须是 `CREATOR`, `public static final` 一个不能少
 - 继承 `Parcelable.Creator` 类
 - 重写2个方法
 - `createFromParcel()`: 从 `Parcel` 容器中取出数据, 并创建一个当前类返回
 - `newArray()`: 固定写法, 在反序列化本类数组的时候调用

```
public class SellerInfoBean implements Parcelable{
```

```
    public long id;
```

```
    public String info;
```

```
    public String name;
```

```
    public List<FoodInfoBean> list;
```

```
    public SellerInfoBean(Parcel parcel){
```

```
        long id1 = parcel.readLong();
```

```
        String[] strs = new String[2];
```

```
        parcel.readStringArray(strs);
```

```
        List<FoodInfoBean> list1 = new ArrayList<>();
```

```
        parcel.readList(list1,
```

```
FoodInfoBean.class.getClassLoader());    // 注意写法
```

```
        id = id1;
```

```
        info = strs[0];
```

```
        name = strs[1];
```

```
        list = list1;
```

```
    }
```

```
@Override
```

```
public int describeContents() {
```

```
    return 0;
```

```
}
```

```
@Override
```

```
public void writeToParcel(Parcel dest, int flags) {
```

```
    dest.writeLong(id);
```

```
    dest.writeStringArray(new String[]{info,name});
```

```

        dest.writeList(list);
    }

    // 泛型是当前类
    public static final Parcelable.Creator CREATOR = new
Parcelable.Creator<SellerInfoBean>(){
        @Override
        public SellerInfoBean createFromParcel(Parcel source)
        {
            SellerInfoBean sellerInfoBean = new
SellerInfoBean(source);
            return sellerInfoBean;
        }

        @Override
        public SellerInfoBean[] newArray(int size) {
            return new SellerInfoBean[size];
        }
    };
}

```

源码

Parcelable 的序列化是依靠 **Parcel** 实现的，最终调用 **writeToParcel()**、**readFromParcel()**、**createFromParcel()** 将内部数据保存到 Parcel 的内存中

- Parcel 有自己的存储格式，所以不同数据需要用不同的方式进行读写

// Parcel

```
public final void writeValue(Object v) {
```

```
    if (v == null) {
```

```
        writeInt(VAL_NULL);
```

```
    } else if (v instanceof String) {
```

//

String

```
        writeInt(VAL_STRING);
```

```
        writeString((String) v);
```

```
    } else if (v instanceof Integer) {
```

//

Integer

```
        writeInt(VAL_INTEGER);
```

```
        writeInt((Integer) v);
```

```
    } else if (v instanceof Map) {
```

//

Map

```
        writeInt(VAL_MAP);
```

```
        writeMap((Map) v);
```

```
    } else if (v instanceof Bundle) {
```

//

Bundle

```
        writeInt(VAL_BUNDLE);
```

```
        writeBundle((Bundle) v);
```

```
    } else if (v instanceof PersistableBundle) {
```

```
        writeInt(VAL_PERSISTABLEBUNDLE);
```

```
        writePersistableBundle((PersistableBundle) v);
```

```
    } else if (v instanceof Parcelable) {
```

//

Parcelable

```
        writeInt(VAL_PARCELABLE);
```

```
        writeParcelable((Parcelable) v, 0);
```

```
    } else if (v instanceof Short) {
```

//

Short

```
        writeInt(VAL_SHORT);
```

```
        writeInt(((Short) v).intValue());
```

```

} else if (v instanceof Long) {
    writeInt(VAL_LONG);
    writeLong((Long) v);
} else if (v instanceof Float) {
    writeInt(VAL_FLOAT);
    writeFloat((Float) v);
} else if (v instanceof Double) {
    writeInt(VAL_DOUBLE);
    writeDouble((Double) v);
} else if (v instanceof Boolean) {
    writeInt(VAL_BOOLEAN);
    writeInt((Boolean) v ? 1 : 0);
} else if (v instanceof CharSequence) {

```

```
//
```

CharSequence

```

    writeInt(VAL_CHARSEQUENCE);
    writeCharSequence((CharSequence) v);
} else if (v instanceof List) {
    writeInt(VAL_LIST);
    writeList((List) v);
} else if (v instanceof SparseArray) {

```

```
//
```

SparseArray

```

    writeInt(VAL_SPARSEARRAY);
    writeSparseArray((SparseArray) v);
} else if (v instanceof boolean[]) {
    writeInt(VAL_BOOLEANARRAY);
    writeBooleanArray((boolean[]) v);
} else if (v instanceof byte[]) {
    writeInt(VAL_BYTEARRAY);
    writeByteArray((byte[]) v);
} else if (v instanceof String[]) {

```

```
//
```

String[]

```

    writeInt(VAL_STRINGARRAY);

```



```
writeStringArray((String[]) v);
} else if (v instanceof CharSequence[]) {
    writeInt(VAL_CHARSEQUENCEARRAY);
    writeCharSequenceArray((CharSequence[]) v);
} else if (v instanceof IBinder) {
    writeInt(VAL_IBINDER);
    writeStrongBinder((IBinder) v);
} else if (v instanceof Parcelable[]) {
    writeInt(VAL_PARCELABLEARRAY);
    writeParcelableArray((Parcelable[]) v, 0);
} else if (v instanceof int[]) {
    writeInt(VAL_INTARRAY);
    writeIntArray((int[]) v);
} else if (v instanceof long[]) {
    writeInt(VAL_LONGARRAY);
    writeLongArray((long[]) v);
} else if (v instanceof Byte) {
    writeInt(VAL_BYTE);
    writeInt((Byte) v);
} else if (v instanceof Size) {
    writeInt(VAL_SIZE);
    writeSize((Size) v);
} else if (v instanceof SizeF) {
    writeInt(VAL_SIZEF);
    writeSizeF((SizeF) v);
} else if (v instanceof double[]) {
    writeInt(VAL_DOUBLEARRAY);
    writeDoubleArray((double[]) v);
} else {
    Class<?> clazz = v.getClass();
    if (clazz.isArray() && clazz.getComponentType() ==
Object.class) {
```

```

        writeInt(VAL_OBJECTARRAY);
        writeArray((Object[]) v);
    } else if (v instanceof Serializable) {
        writeInt(VAL_SERIALIZABLE);
        writeSerializable((Serializable) v);
    } else {
        throw new RuntimeException("Parcel: unable to marshal
value " + v);
    }
}
}
}

```

```

public final void writeParcelable(Parcelable p, int
parcelableFlags) {
    if (p == null) {
        writeString(null);
        return;
    }
    writeParcelableCreator(p);
    p.writeToParcel(this, parcelableFlags);
    // writeToParcel()
}

```

Parcel

参考 [【05】 Binder Data – Parcel.md](#)

Parcel 是进程间传递数据的方式，拥有 **数据打包和重组** 的能力

- 溢出：如果超出了 Parcel 的存储能力，它会自动申请所需的内存，并扩展 dataSize、dataCapacity
- 原理：Parcel 底层采用 C 结构体的方式对数据进行了重新打包和重组
- 读写是配套的：用什么方式写，就需要用什么方式取

Parcel 是一个扁平的 "**二进制数据容器**"，它可以存储各种类型的数据，并且能够跨进程传输

Parcel 底层相当于，按照一定规则对数据重新进行了组装（二进制数据）；然后读取的时候，也按照对应规则进行读取

```
dataSize();           // 容量
dataCapacity();       // 最大容量
dataAvailable();      // 剩余大小
dataPosition();       // 读写位置

// 读写

// 基本数据类型
writeDouble(double value);
readDouble();

writeByte(byte value);
readByte();

writeByteArray(byte[] value, int start, int count); // 先写
入4个字节表示数据大小，再写入数据本身
readByteArray(byte[] value);
byte[] createByteArray();
```

// List、Array

```
writeArray(Object[] value);  
writeList(List value);  
readArray();  
readList();
```

// Parcelable

```
writeParcelable(Parcelable value);
```

// 实际上是

回调 Parcelable.writeToParcel 实现的

```
readParcelable(ClassLoader);  
writeParcelableArray(T[], int);  
readParcelableArray(ClassLoader);
```

// Bundle

```
writeBundle(Bundle value);  
readBundle();
```

// Active Object（当读取时，并不是重新创建一个新的，而是原来的对象）

```
writeStrongBinder(Binder value);  
writeStrongInterface(IInterface value);  
readStrongBinder();  
  
writeFileDescriptor(FileDescriptor value);  
readFileDescriptor();
```

序列化的应用

对象转字符串

序列化：目的是使对象能够通过 IO 流进行传递

- 对象转字节（序列化）：`ByteArrayOutputStream` + `ObjectOutputStream`
- 字节转对象（反序列化）：`ByteArrayInputStream` + `ObjectInputStream`

16 进制：字符串一般采用 ASCII 编码，但是有时候会造成乱码，如果用 HEX16 进制数表示，就不会造成乱码了

- Using some super basic byte array <-> hex conversions so we don't have to rely on any* large Base64 libraries. Can be overridden if you like!

```
/**
 * 对象转字符串
 */
public String encodeCookie(Object obj) {
    if (obj == null)
        return null;
    try {
        // 1. 序列化
        SerializableObject serializableObj = new
SerializableObject(obj);
        // 2. 对象转字节流
        ByteArrayOutputStream baos = new
ByteArrayOutputStream();
```

```

        ObjectOutputStream oos = new
ObjectOutputStream(baos);
        oos.writeObject(serializableCookie);
        // 3. 字节流转字节数组
        byte[] bytes = baos.toByteArray();
        // 4. 字节数组转字符串
        return bytes2Hex(bytes);
    } catch (IOException e) {
        e.printStackTrace();
        return null;
    }
}

/**
 * 字符串转对象
 */
public Object decodeCookie(String str) {
    if (TextUtils.isEmpty(str))
        return null;
    try {
        // 1. 字符串转字节数组 (16进制)
        byte[] bytes = hex2Bytes(str);
        // 2. 字节数组转IO流
        ByteArrayInputStream bis = new
ByteArrayInputStream(bytes);
        ObjectInputStream ois = new ObjectInputStream(bis);
        // 3. 字节流转对象 (序列化)
        SerializableCookie serializableCookie =
(SerializableCookie) ois.readObject();
        return serializableCookie.getCookie();
    } catch (Exception e) {
        e.printStackTrace();
    }
}

```

```
return null;
```

```
}
```

```
}
```

```
// 字节数组转字符串 (2进制)
```

```
private String bytes2Hex(byte[] bytes) {
```

```
    StringBuilder sb = new StringBuilder(bytes.length * 2);
```

```
    for (byte element : bytes) {
```

```
        int v = element & 0xff;
```

```
        if (v < 16) {
```

```
            sb.append('0');
```

```
        }
```

```
        sb.append(Integer.toHexString(v));
```

```
    }
```

```
    return sb.toString().toUpperCase(Locale.US);
```

```
}
```

```
// 字符串转字节数组 (16进制)
```

```
private byte[] hex2Bytes(String hexString) {
```

```
    int len = hexString.length();
```

```
    byte[] data = new byte[len / 2];
```

```
    for (int i = 0; i < len; i += 2) {
```

```
        data[i / 2] = (byte)
```

```
((Character.digit(hexString.charAt(i), 16) << 4) +
```

```
Character.digit(hexString.charAt(i + 1), 16));
```

```
    }
```

```
    return data;
```

```
}
```

Intent、Bundle

Intent 和 **Bundle** 都能传递序列化参数

- Serializable 只能传递具体的对象（8大基本数据类型、String、Serializable、Parcelable），ArrayList 本身实现了 Serializable 接口
- Parcelable 能传递具体的对象、**数组**、**集合**

bundle/Intent 不能传递大于 40K 的数据（可以用一个 单例对象\EventBus 进行传递）

- 本质是利用 Linux 共享内核进行传递数据，而这个共享内核有一个限制
- 理论大小是 **1M-8K**，但是实际大小是 **512K**

传 512K 以下的数据的数据可以正常传递

传 512K~1024K 的数据有可能会出错，闪退

传 1M以上的数据会报错：TransactionTooLargeException

考虑到 Intent 还包括要启动的 Activity 等信息，实际可以传的数据 **略小于 512K**


```
intent.putExtra(String key, Serializable value)
intent.putExtra(String key, Parcelable value)
intent.putExtra(String key, Parcelable[] value)    // 数组
intent.putParcelableArrayListExtra(String key, ArrayList<?
extends Parcelable> value)    // 集合

bundle.putSerializable(String key, Serializable value)
bundle.putParcelable(String key, Parcelable value)
bundle.putParcelableArray(String key, Parcelable[] value)
// 数组
bundle.putParcelableArrayList(String key, ArrayList<? extends
Parcelable> value)    // 集合
```

底层

Intent、Bundle 都实现了 Parcelable 接口，可以用来进行进程间传递数据，但是它们的功能又略有不同

- 有些 Intent 传递不了的对象（例如 **IBinder**），可以借助 Bundle 来实现

Parcel：Intent 和 Bundle 底层都是借助 Parcel 实现的，因此它们有的功能 Parcel 也有

```
val binder = IBinder()
```

```
// 将 IBinder 对象放入 Bundle
```

```
val bundle = Bundle()
```

```
bundle.putBinder("binderKey", binder)
```

```
// 将 Bundle 放入 Intent
```

```
val intent = Intent(context, TargetActivity::class.java)
```

```
intent.putExtra("bundleKey", bundle)
```

```
startActivity(intent)
```

对进程间通讯的思考

这里有个问题，我们跨进程通讯的时候，

1. 如果 2 个进程是同一个进程（例如 A->B->A），那么使用 Parcel 传递的 IBinder 对象，可以强转为目标对象
2. 如果 2 个进程是不同进程（例如 A->B），那么使用 Parcel 传递的 IBinder 对象，在强转的时候会报错，只能使用 AIDL.Stub.asInterface() 转化为 Proxy 代理类
3. 使用 Intent 传递 callback 回调函数的时候，居然能调用到同一个对象里来

原因（IBinder 不能强转的原因）：

1. 这是因为每个进程都有自己 独立的虚拟地址空间，它们的内存地址并不重叠
2. 这样同一个类，虽然它们的“内存结构”相同，但是它们的

“类信息”是存储在 2 个“不同的内存地址”中的，被识别为“2 个不同的类”

3. 当强转的时候，一个是 A 内存中的类，一个是 B 内存中的类，就会报错

原因（IBinder 同进程可以强转的原因）：

1. 虽然 Parcel 对数据进行了打包重组，但是它们的内存结构还是一样的
2. 当从 A->B->A 回来的时候，它们还是指向的同一个内存地址（类型一样、对象实例一样、内存地址一样）
3. 所以我们可以 直接强转、直接调用目标方法、直接使用数据

原因（AIDL 可以跨进程的原因）

1. IBinder 对象在内存中有一个 映射关系，它们可以找到彼此
2. 在调用 `AIDL.Stub.asInterface()` 转化为 Proxy 代理类的时候，它并不是真的强转，而是对 IBinder 进行了包装，内部还是借助 IBinder 来实现的
3. AIDL 文件只是为了 统一两端的方法调用问题（约定规则）